McGill

# Functions, Recursion, and While Loops

Deeksha Arya

# Recall

- We learned that functions can *return* values using the `return` keyword.

- The `return` keyword sends back the value that we put after the keyword to wherever it was called.

```
def function_name(args):
    return something
```

- Write a function `add()` that takes a and b as input and returns the sum or a and b.

# Recall

- We learned that functions can *return* values using the `return` keyword.

- The `return` keyword sends back the value that we put after the keyword to wherever it was called.

```
def function_name(args):
    return something
```

```
def add(a, b):
    return a + b
```

# Recall

- In Python, we can have local and global variables.

```
def add(a, b):
    return a + b
```

- a and b are local variables, because they exist only within the scope of the function.

- Global variables exist throughout the scope of our file. Typically, constants are global variables.

# Recall

- Wherever possible, it is better to take values as input, manipulate them as local variables, and return them to the main program, rather than manipulate global variables directly. This is so we do not lose track of when our global variables have been changed.

# Recall

- A DocString provides information within triple quotes (""") at the beginning of a function about:
  - The functions' input arguments and their types
  - What the function does
  - The functions' return values and their types
  - Some worked examples of input values and return values
- Write the DocString for your function `add(a, b)`

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b
```

- How do we return all these values?

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi
```

We can return multiple values from a python function by separating them with commas.

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi
```

- How can we call this function and save its return values?

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi
```

- How can we call this function and save its return values?

```
m, n, p, q = calculator(10, 20)
```

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi
```

The input arguments are taken in order.

```
m, n, p, q = calculator(10, 20)
```

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi
```

The output arguments are also taken in order. This is a multiple assignment statement.

```
m, n, p, q = calculator(10, 20)
```

# Functions with `return`

- Write a function called `calculator(a, b)` to calculate the sum, difference, multiplication and division of a and b.

The order of input arguments and the output values are **important.**

```
def calculator(a, b):
    summ = a + b
    diff = a - b
    mult = a * b
    divi = a / b

    return summ, diff, mult, divi

m, n, p, q = calculator(10, 20)
```

# Recursive Functions

- It is possible to have a function that calls itself. This is called a recursive function.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

# Recursive Functions

- It is possible to have a function that calls itself. This is called a recursive function.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

A recursive function has a base condition.

- When n is 0, the function prints “Blastoff!”. This is the base condition or base case. After the base condition, the control flow returns to where the function was called.

# Recursive Functions

- It is possible to have a function that calls itself. This is called a recursive function.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

A recursive function calls itself with a changed input.

- When n is not 0, the function prints n and then calls itself with n-1. By changing the input, the function “moves towards” achieving the base condition.

# Recursive Functions

- For a finite recursive function:
  - A recursive function must have a base case / condition.
  - The function must change its state, e.g. a variable on which the base condition depends.
  - The function must call itself with its new state.

- Source: https://runestone.academy/ns/books/published/pythonds/Recursion/TheThreeLawsofRecursion.htm

# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

- What happens if we call the function:

countdown(3)

# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
```

Recursion steps:

n = 3

# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
```

Recursion steps:

n = 3

# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
    3
```

Recursion steps:

n = 3

# Recursive Functions

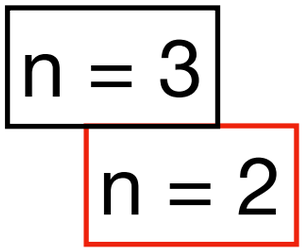
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
    3
```

Recursion steps:



# Recursive Functions

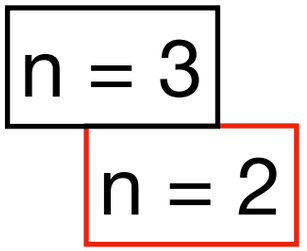
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
    3
```

Recursion steps:



# Recursive Functions

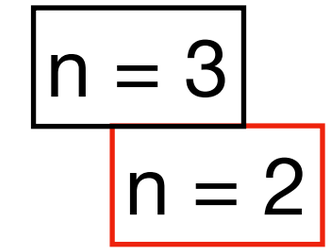
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

Output:

```
>>> %Run rocketship.py
3
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

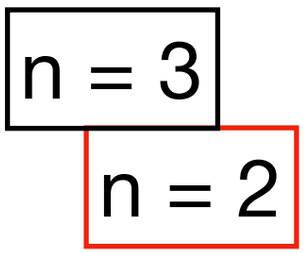
countdown(3)
```



Output:

```
>>> %Run rocketship.py
    3
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

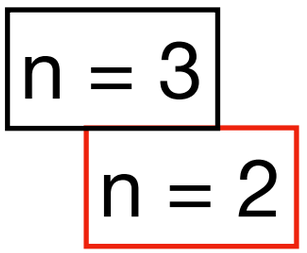


Output:

```
>>> %Run rocketship.py

   3
   2
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

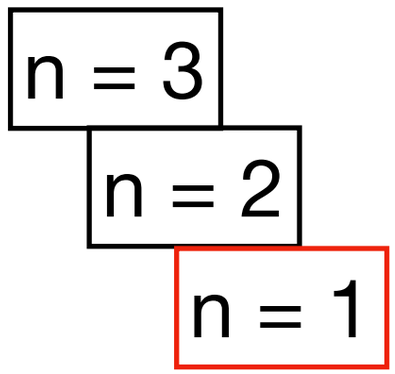
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

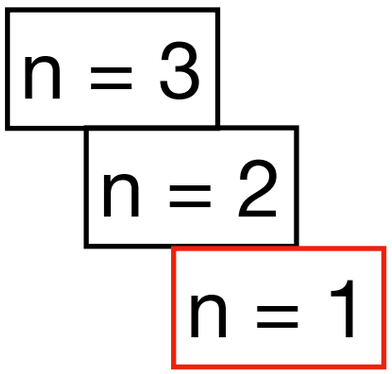
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

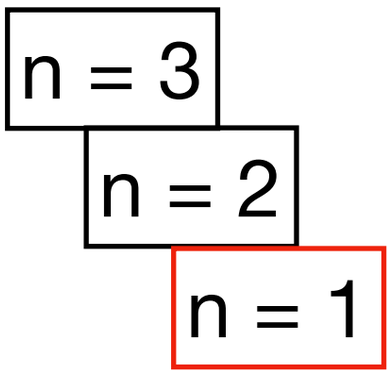
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

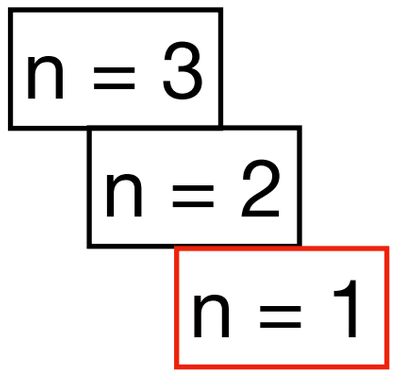


Output:

```
>>> %Run rocketship.py

   3
   2
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

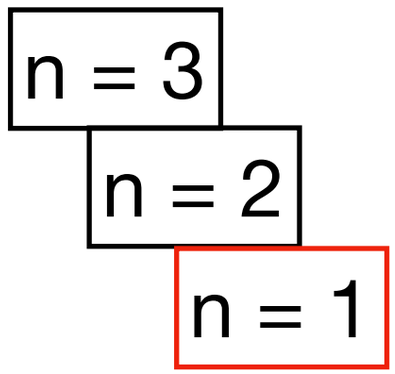
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

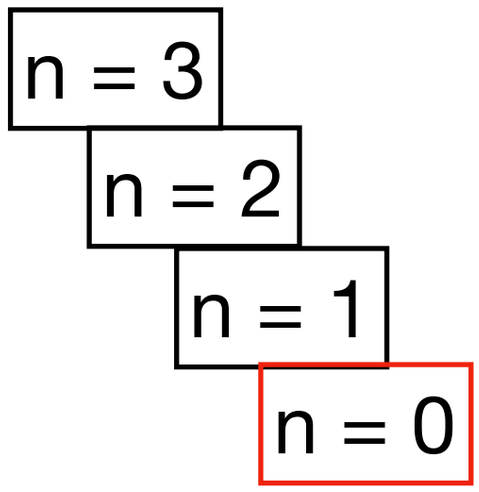
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

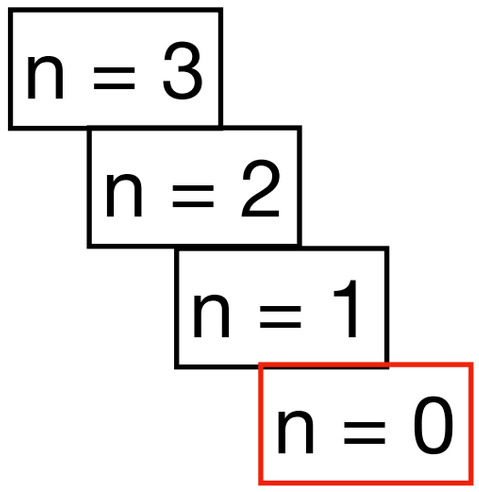
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

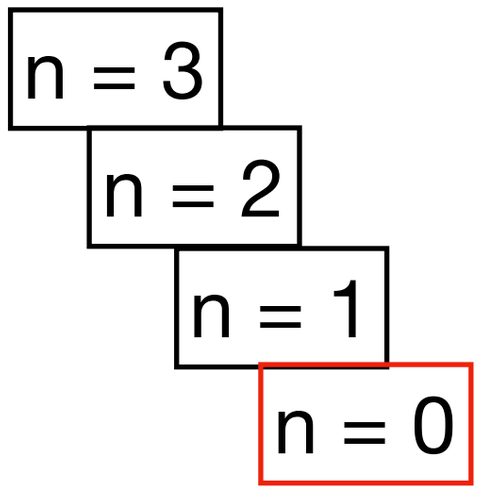
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)

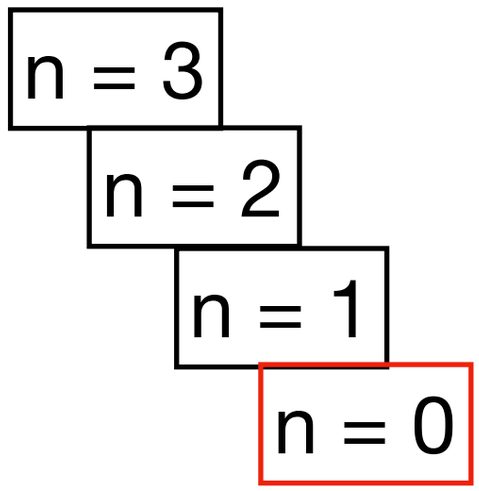
countdown(3)
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
   Blastoff!
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
countdown(3)
```

The function returns to where it was called.

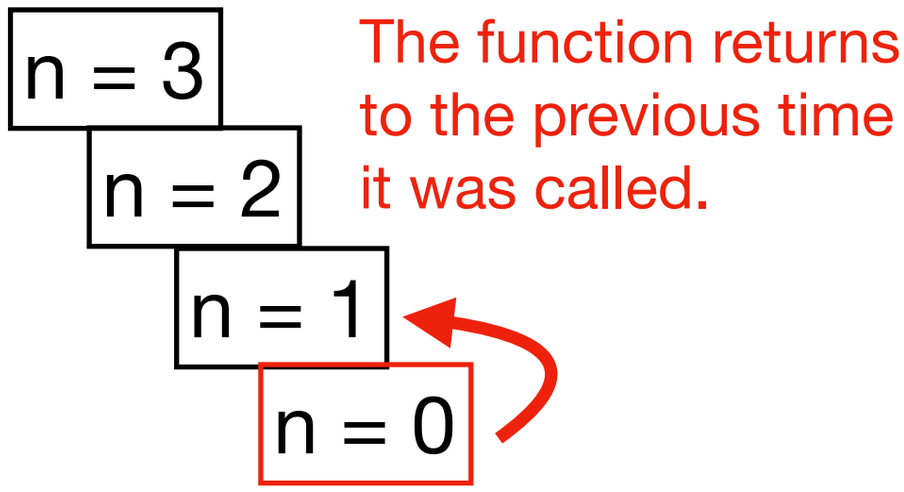
It was called on this line when n was 1.

## Output:

```
>>> %Run rocketship.py

   3
   2
   1
   Blastoff!
```

## Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
countdown(3)
```

There is nothing to run after this function call, so the control flow returns to where the function was called:

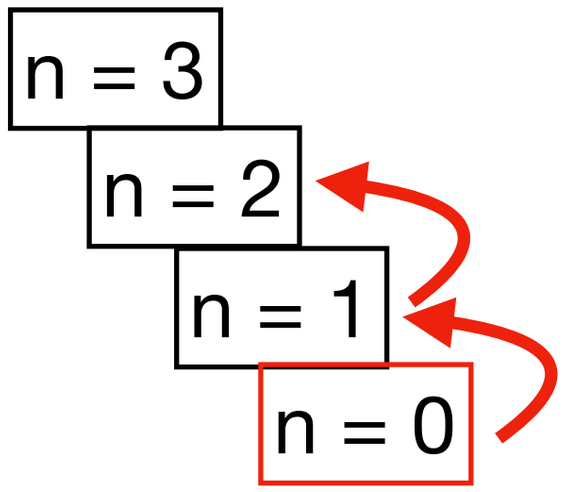
i.e. the same line but when n was 2

Output:

```
>>> %Run rocketship.py

   3
   2
   1
   Blastoff!
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
countdown(3)
```

There is nothing to run after this function call, so the control flow returns to where the function was called:

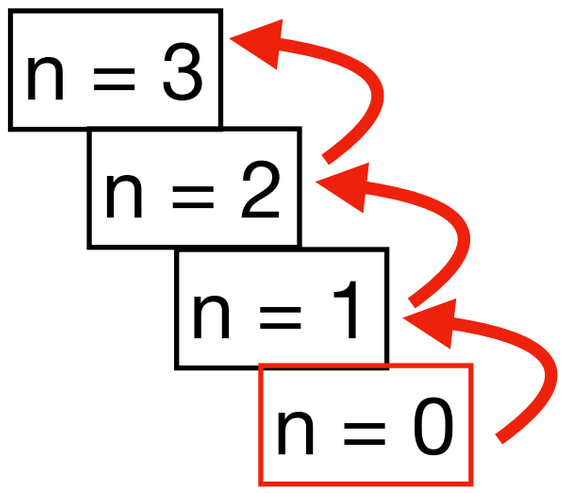
i.e. the same line but when n was 2

Output:

```
>>> %Run rocketship.py

   3
   2
   1
   Blastoff!
```

Recursion steps:



# Recursive Functions

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

→ `countdown(3)`

There is nothing to run after this function call, so the control flow returns to where the function was called:

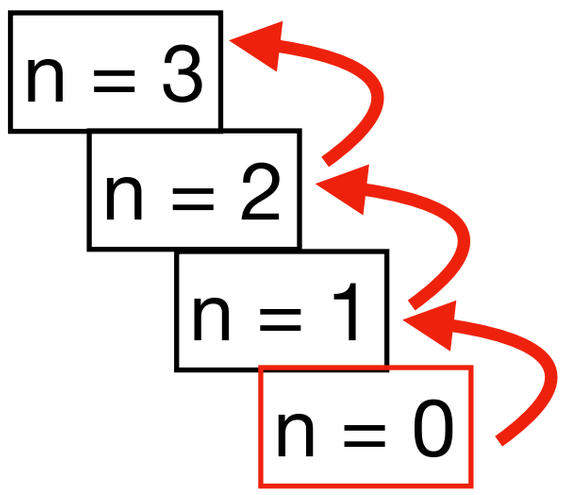
i.e. the first call to the function

## Output:

```
>>> %Run rocketship.py

   3
   2
   1
   Blastoff!
```

## Recursion steps:



# Recursive Algorithm

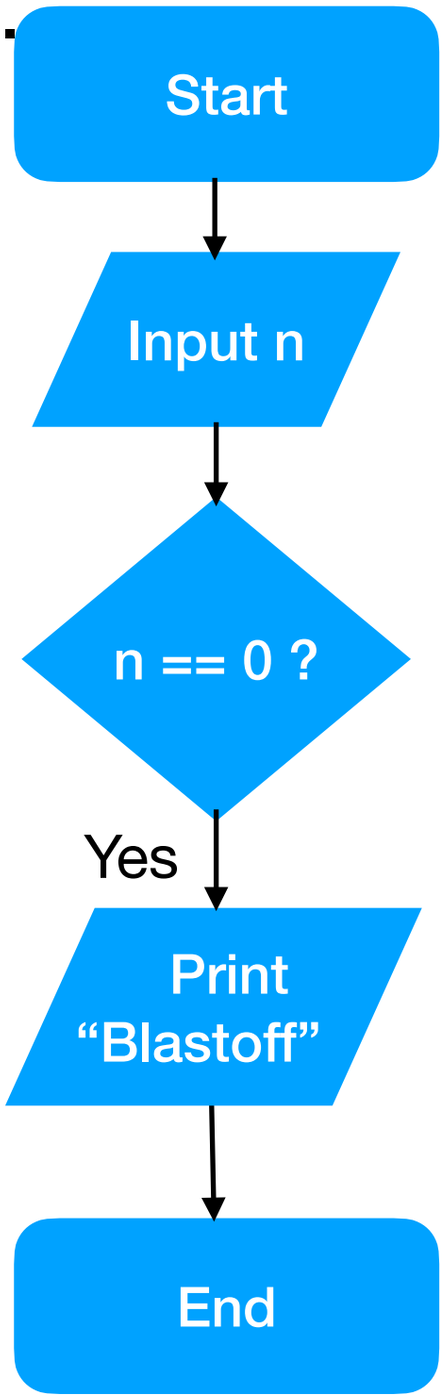
To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

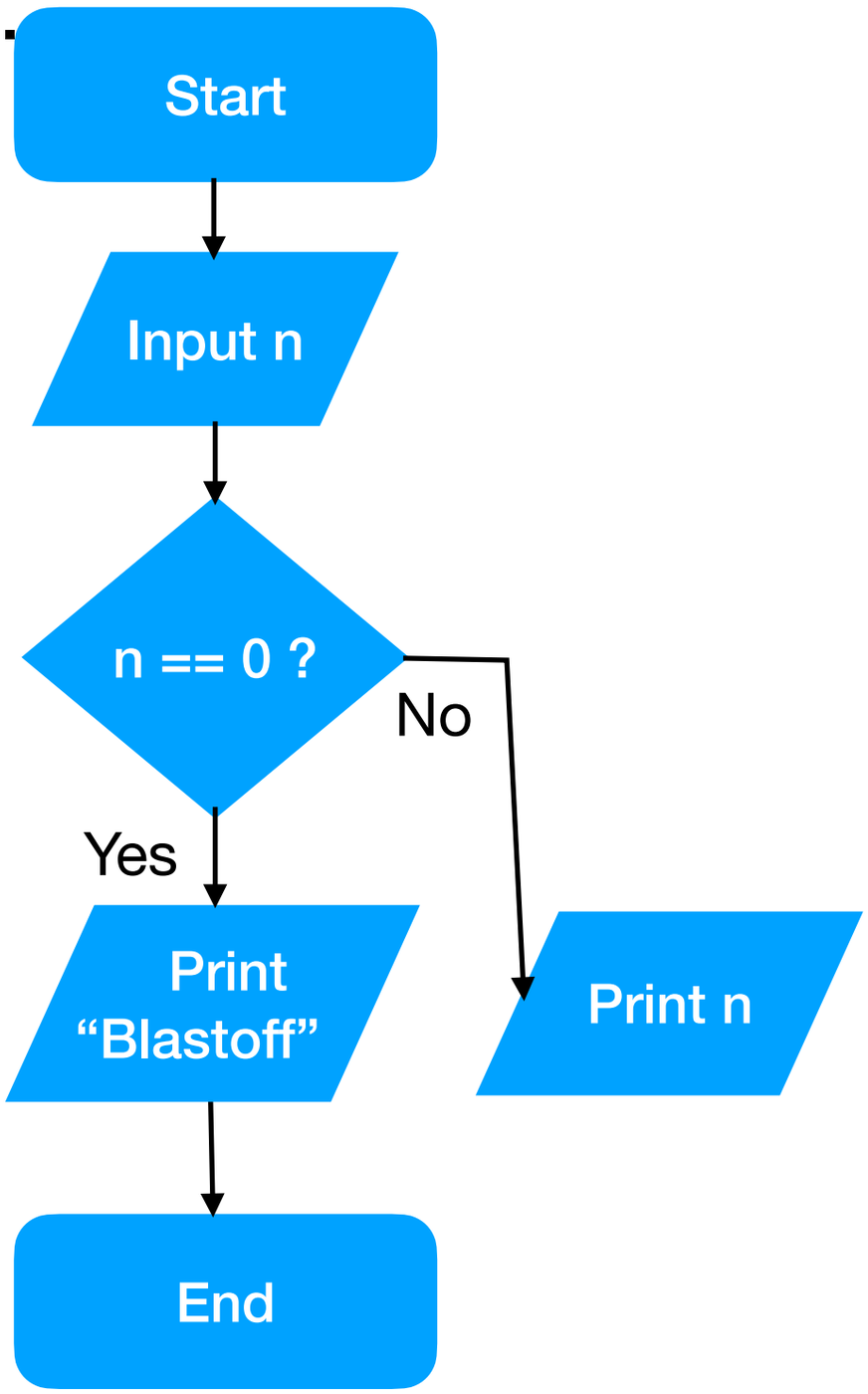
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```



# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

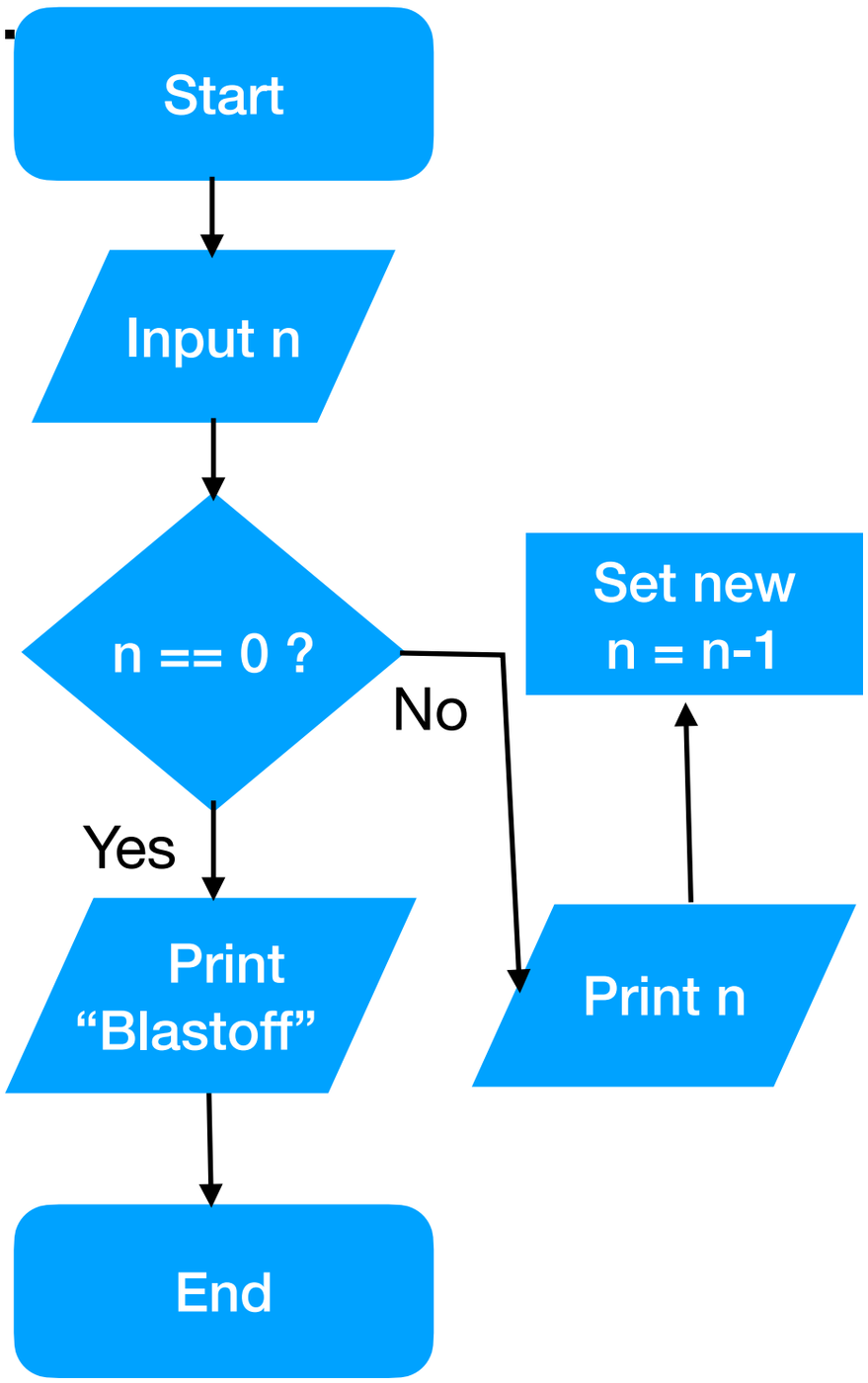
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```



# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

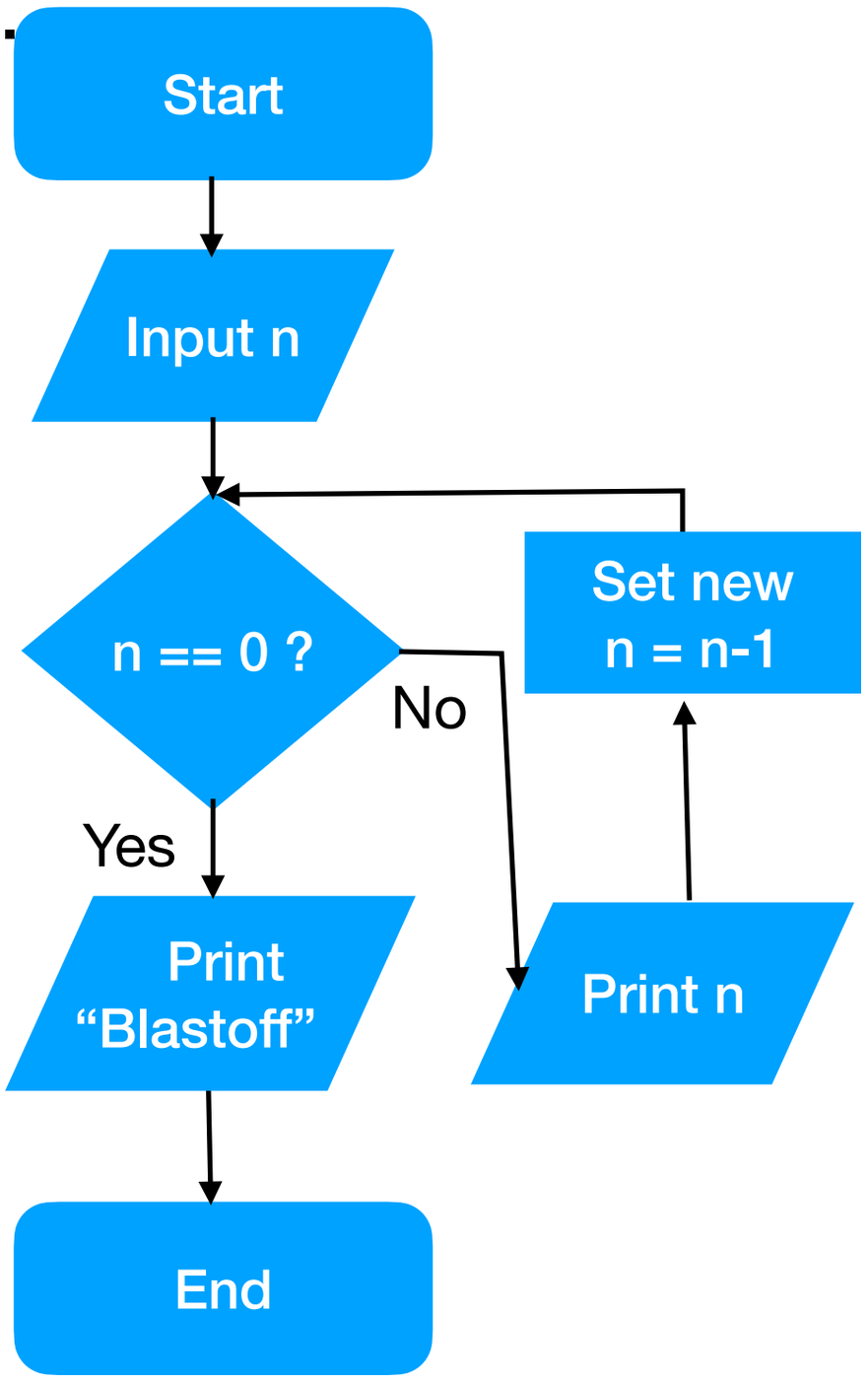
```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```



# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

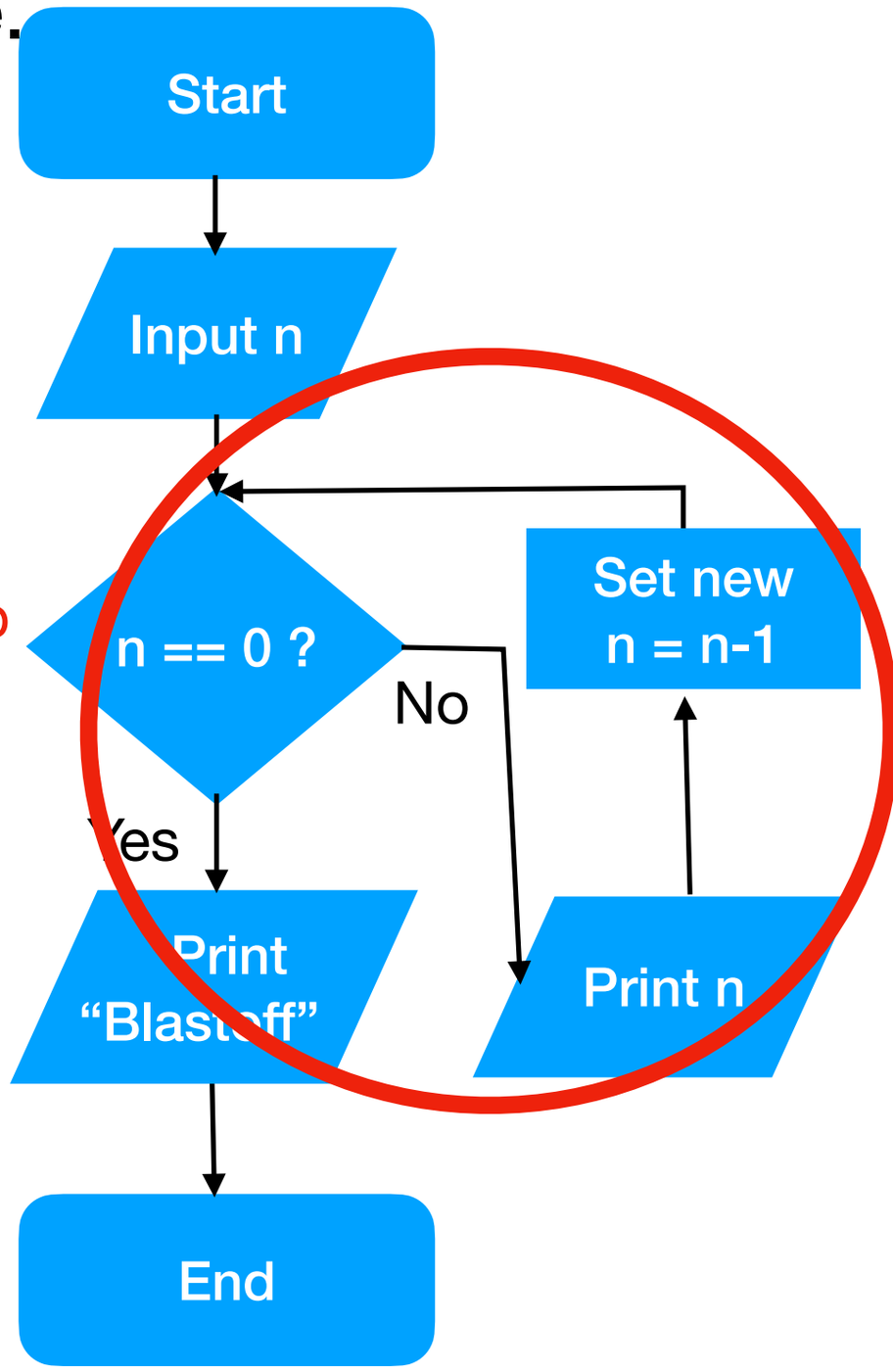


# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

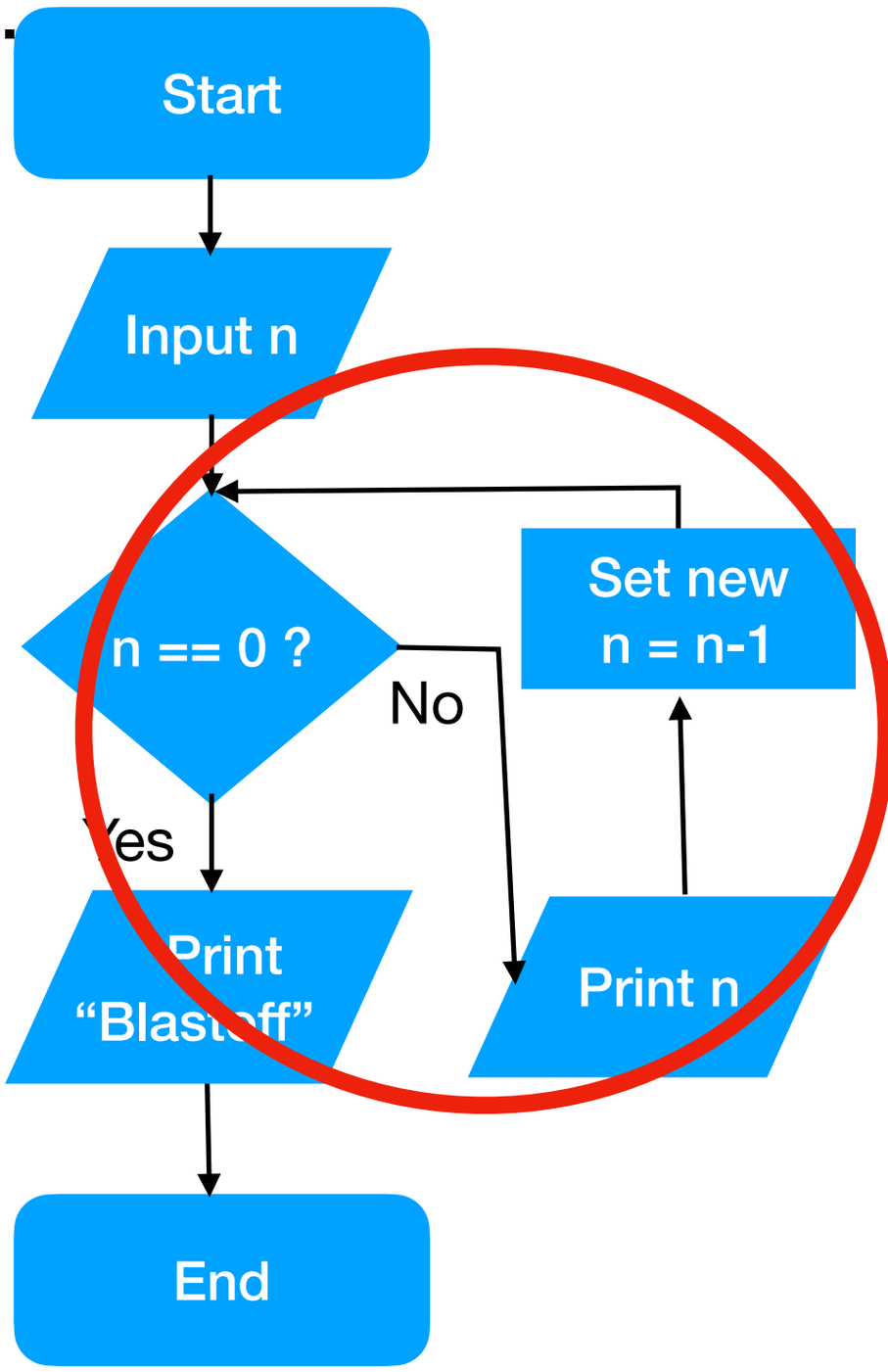
This forms a loop that continues until the base condition is satisfied.



# Recursive Algorithm

To understand what just happened a bit better, let's draw a flowchart representing the algorithm behind this code.

```
def countdown(n):
    if n == 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```




Notice the loop in the flow chart capture only the calls in recursion, not the return steps!

# Recursive Functions

- Update your countdown program to print(n) after the function call as well. What is the output? Why is this the output?

```
def countdown(n):
    if n == 0:
        print('Blastoff')
    else:
        print(n)
        countdown(n-1)
        print(n)

countdown(3)
```

# Recursive Functions

- Update your countdown program to print(n) after the function call as well. What is the output? Why is this the output?

```
def countdown(n):
    if n == 0:
        print('Blastoff')
    else:
        print(n)
        countdown(n-1)
        print(n)

countdown(3)
```

```
>>> %Run rocketship.py

  3
  2
  1
  Blastoff
  1
  2
  3
```

# Control Flow

- The **control flow** of a program determines:
  - Which part of the code should always be executed,
  - Which part of the code should be executed **only under certain circumstances,** (if-elif-else)
  - Which parts of the code should be executed **repeatedly**, and how many times it should be repeated. (Called *iteration* with *loops*, which we will see today!)

# Loops

- While recursive functions allow us to simulate a loop, they are not efficient in terms of time and computation, because there are multiple “steps” for the loop to occur.

- As an alternative, programming languages like Python provide ways to create loops. This is referred to as *iteration*.

# While Loops

- Syntax:

```
while condition:
        instruction 1
        instruction 2
        etc
```

The block of code is repeatedly executed as long as the condition evaluates to **True**.

# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

# While loops

- To write a program that runs a countdown using a while loop:

The condition, a boolean expression

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

# While loops

- To write a program that runs a countdown using a while loop:

The condition, a boolean expression

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

The loop body (Notice the indent)

# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

The condition, a boolean expression

The loop body (Notice the indent)

Inside the loop, we have an update statement that moves us towards satisfying the loop condition, so that we can eventually exit the loop.

# While loops

- To write a program that runs a countdown using a while loop:

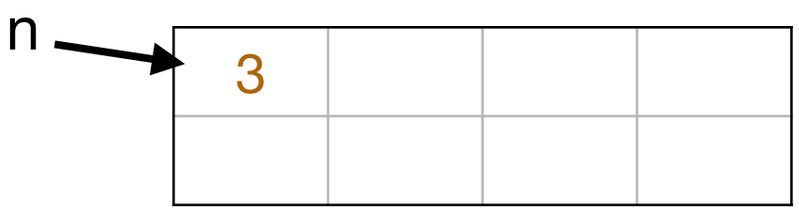
```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

Output:

```
>>> %Run rocketship.py
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

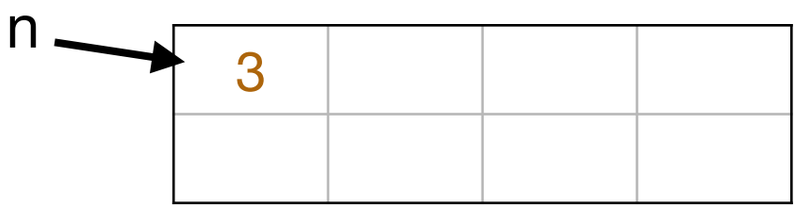
print('Blastoff!')
```

Yes, n != 0,
go in the loop

Output:

```
>>> %Run rocketship.py
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

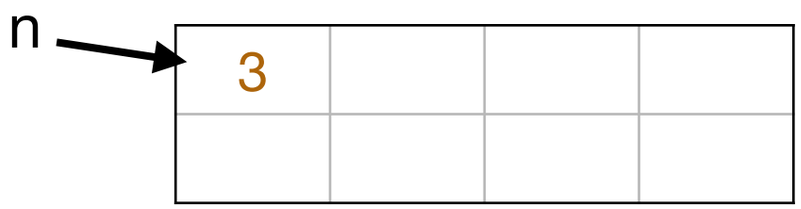
print('Blastoff!')
```

## Output:

```
>>> %Run rocketship.py

   3
```

## Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

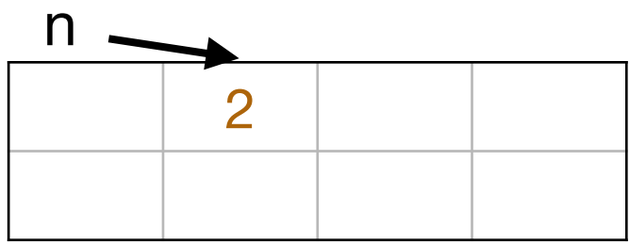
print('Blastoff!')
```

## Output:

```
>>> %Run rocketship.py

  3
```

## Variables:



# While loops

- To write a program that runs a countdown using a while loop:

At the end of the loop block, we go back and check the condition again.

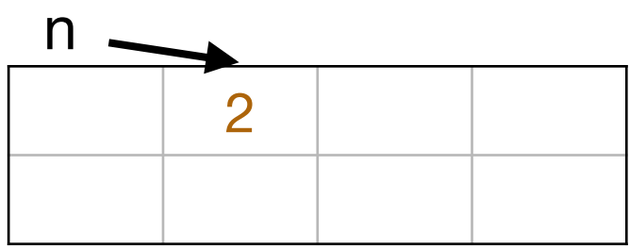
```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1
print('Blastoff!')
```

Yes, n != 0, go in the loop

## Output:

```
>>> %Run rocketship.py
   3
```

## Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

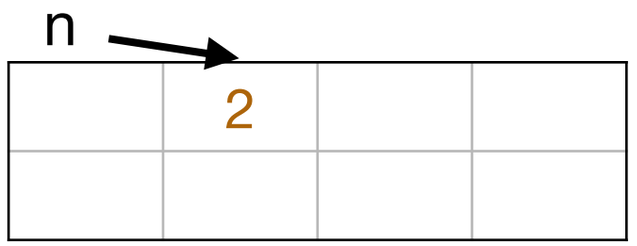
print('Blastoff!')
```

Output:

```
>>> %Run rocketship.py

   3
   2
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

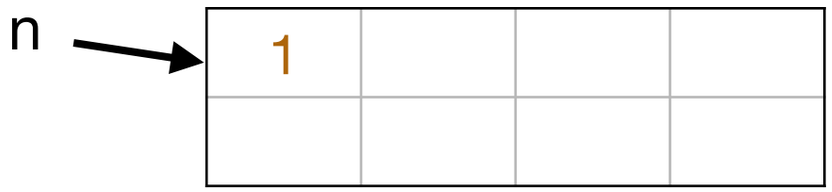
print('Blastoff!')
```

Output:

```
>>> %Run rocketship.py

   3
   2
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

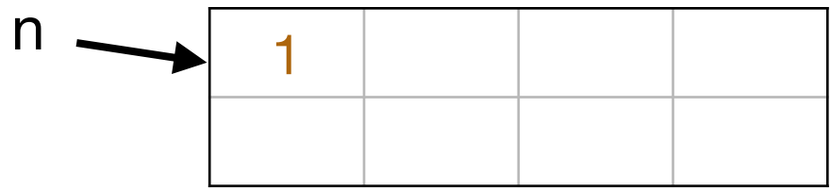
Yes, n != 0,
go in the loop

Output:

```
>>> %Run rocketship.py

  3
  2
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

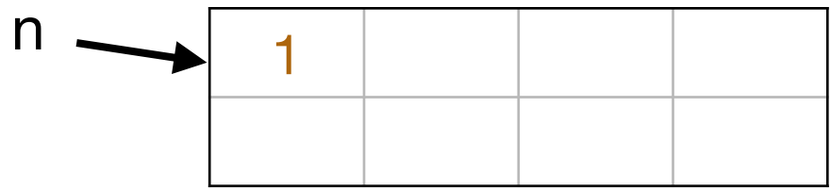
print('Blastoff!')
```

## Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

## Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

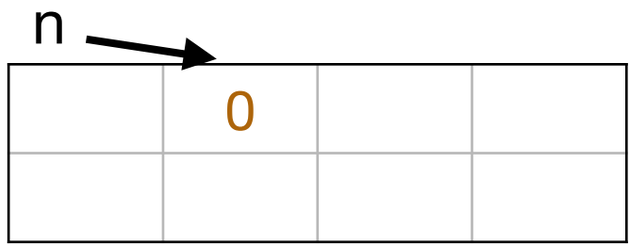
print('Blastoff!')
```

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

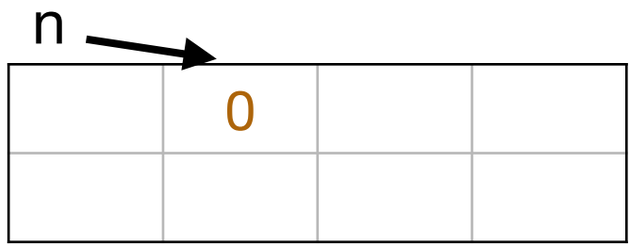
No, n == 0,
so don't go in the loop!

Output:

```
>>> %Run rocketship.py

   3
   2
   1
```

Variables:



# While loops

- To write a program that runs a countdown using a while loop:

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

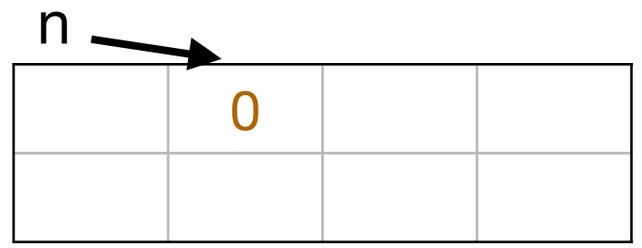
print('Blastoff!')
```

## Output:

```
>>> %Run rocketship.py

  3
  2
  1
  Blastoff!
```

## Variables:



# While loops

- An *iteration* is a single execution of the instructions in the body of the loop.

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

- This code has 3 iterations, i.e. repetitions. The loop block ran three times.

- n acts as a loop *counter* which tells us how many times the loop should run. In this case, we are counting down.

# While loops

- Rewrite the program to count from 1 to 3 (and print these numbers), and then print “Go!”

```
n = 3
while n != 0:
    print(n)
    n = n - 1 # Or shorthand: n -= 1

print('Blastoff!')
```

# If vs While

### `if` statements

- The condition is checked **once**, before executing the block of code (body).
- The block of code is executed **at most once**, **if** the condition evaluates to `True`.

### `while` loops

- The condition is checked **once per iteration of the loop**, before executing the block of code (body).
- The block of code is executed **repeatedly**, **as long as** the condition evaluates to `True`.

# While loops

- What does the following code do?

```
n = 0
while n < 4:
    print(n)
```

# While loops

- What does the following code do?

```
n = 0
while n < 4:
    print(n)
```

- This code creates an infinite loop.

- The loop block will be executed forever since the loop condition can never be satisfied!

- We have to be careful when writing a loop! It is important to make sure that it can eventually terminate.

# Shorthand Assignment

- Python provides a convenient shorthand that combines assignment and arithmetic operators into one operator.

Instead of typing:

```
x = 10
x = x + 7
```

We can type:

```
x = 10
x += 7 # This is the same as x = x+7
```

# Shorthand Assignment

- Python provides a convenient shorthand that combines assignment and arithmetic operators into one operator.
  - `x += y` is equivalent to `x = x + y`
  - `x -= y` is equivalent to `x = x - y`
  - `x *= y` is equivalent to `x = x * y`
  - `x /= y` is equivalent to `x = x / y`
  - `x **= y` is equivalent to `x = x ** y`
  - `x %= y` is equivalent to `x = x % y`
  - `x //= y` is equivalent to `x = x // y`

# Random

- In Python, we can generate (pseudo)-random numbers.

- One way to do this is by using the **random** module. To use it, we must import it at the top of our file:

**import** random

- The module provides different functions (check out the documentation if you are interested)
- For the moment we will focus on :
- **random()** – It returns a random float value between 0.0 (inclusive) and 1.0 (exclusive)
- **randint(**x, y**)** – It returns a random int value between x and y, both included

# Random

**Exercise**:

- Write a function that displays a random float between 0 and 10.
- Modify it so that it returns a random int between 0 and 10.
- Modify it again so that it returns a random int between min and max, where min and max are inputs to the function.

# Random Seed

- The number generation is not truly random. It is determined by an initial value that is called the seed.

- If we don't fix the value of seed, then the value used to generate the numbers is the current system time. This means that each time we run the program, the seed will be different, and the number generated will also be different.

- We can fix the seed using the function **random.seed()**.

# Random Seed

- This program will generate the same 5 integers no matter how many times we run it.
- The integers generated in this case are: 7 35 12 99 53.

```
import random
random.seed(123)
i = 0
while i < 5:
    print(random.randint(1,100))
    i += 1
```

You can read more about random seeds here: https://pynative.com/python-random-seed/ (optional)

# Exercise

- Write a recursive function get_sum_rec(n) that returns the sum of all numbers from 1 to n.

- Write another function get_sum_iter(n) that returns the sum of all numbers 1 to n using a while loop.

- Generate a random number between 1 to 100, and call both your functions with this number. Print whether the two values from the two functions are the same or different.

# References and Acknowledgements

- Thank you to Faten M'hiri, Giulia Alberini, and Jonathan Campbell for the content on some of the slides.

- ThinkPython: https://allendowney.github.io/ThinkPython/chap05.html, https://allendowney.github.io/ThinkPython/chap07.html